正则表达式

一、简介

1、示例 1

2、示例 2

二、语法

1、普通字符

2、非打印字符

3、特殊字符

4、限定符

5、定位符

6、选择

①、括号 ()

②、?=、?<=、?!、?<! 的使用区别

7、反向引用

三、

四、

五、

六、

七、

八、

九、

十、

十一、

十二、

十三、

十四、

十五、

十六、

十七、

十八、

十九、

二十、

1、

2、

3、

4、

5、

6、

7、

8、

9、

①、

②、

③、

④、

⑤、

⑥、

⑦、

⑧、

⑨、

⑩、

Ⅰ、

Ⅱ、

Ⅲ、

Ⅳ、

Ⅴ、

Ⅵ、

Ⅶ、

Ⅷ、

Ⅸ、

Ⅹ、

> 菜鸟教程：https://www.runoob.com/regexp/regexp-syntax.html

一、简介

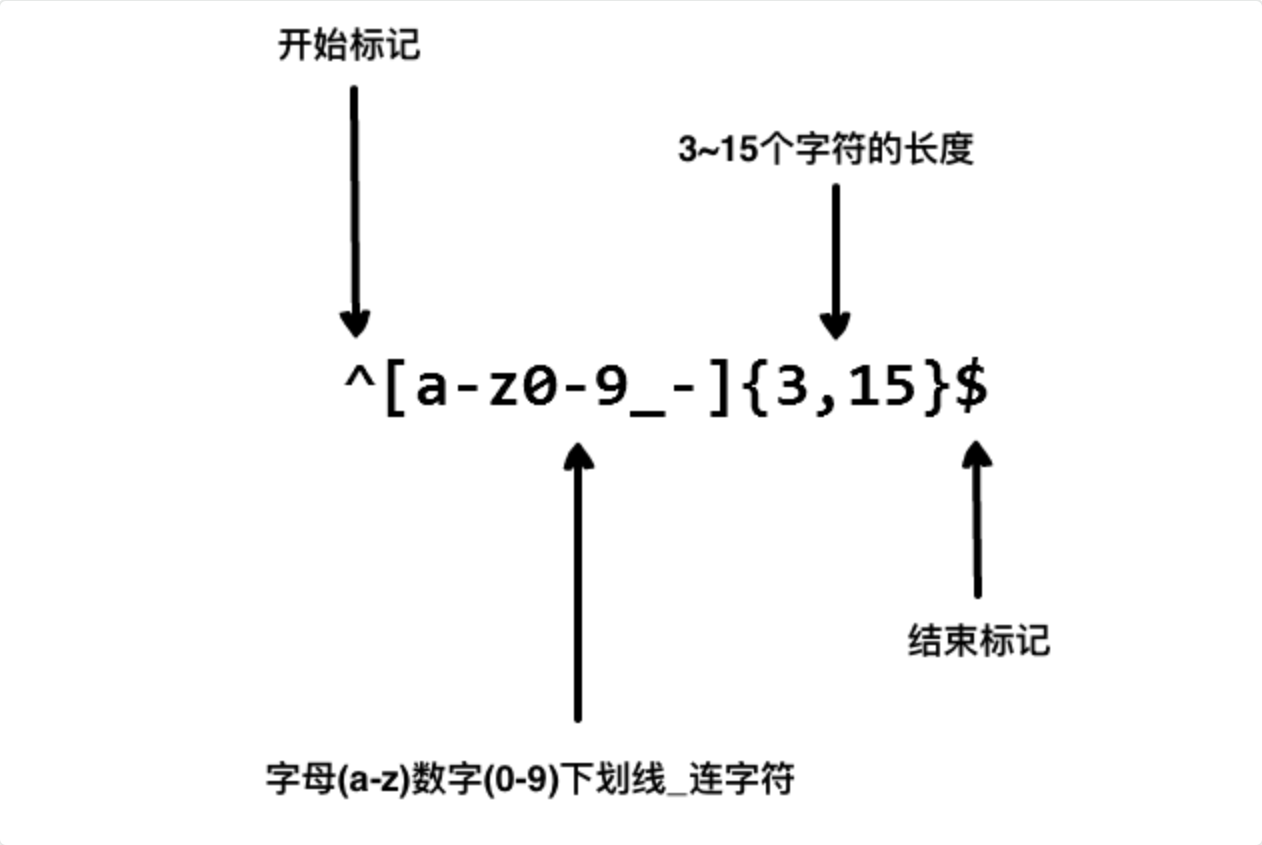
一种文本模式，包括普通字符（例如，a 到 z 之间的字母）和特殊字符（称为"元字符"），是计算机科学的一个概念。正则表达式使用单个字符串来描述、匹配一系列匹配某个句法规则的字符串，通常被用来检索、替换那些符合某个模式（规则）的文本

1、示例 1

```
^[0-9]+abc$
```

- `^` 为匹配输入字符串的开始位置。
- `[0-9]+`匹配多个数字， `[0-9]` 匹配单个数字，`+` 匹配一个或者多个。
- `abc$`匹配字母 `abc` 并以 `abc` 结尾，`$` 为匹配输入字符串的结束位置。

2、示例 2



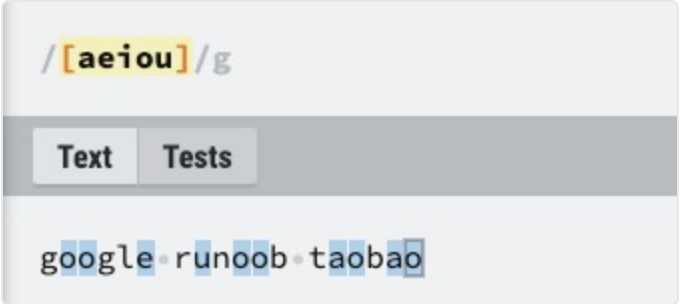
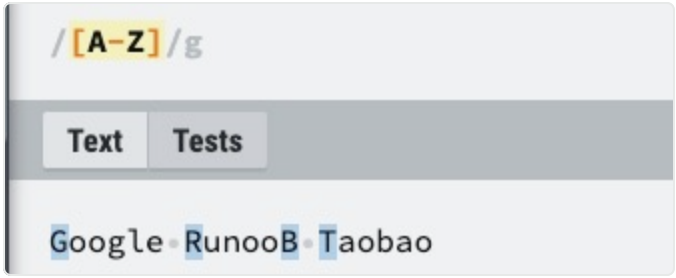
- 只允许用户名包含字符、数字、下划线和连接字符 `-`，并设置用户名的长度

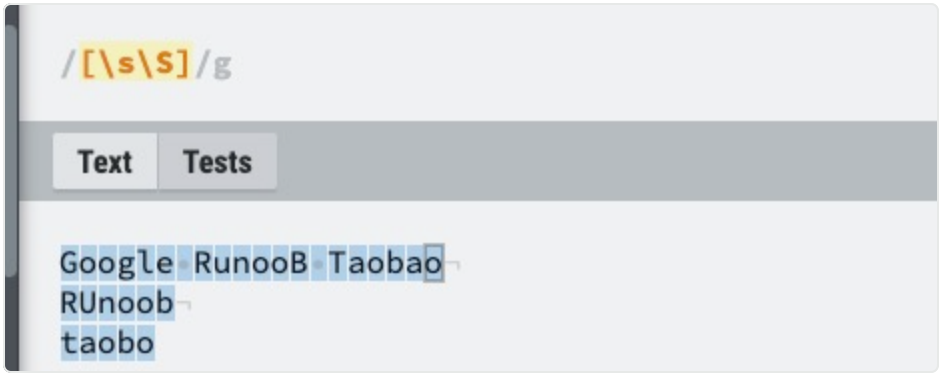
二、语法

1、普通字符

普通字符包括没有显式指定为元字符的所有可打印和不可打印字符。这包括所有大写和小写字母、所有数字、所有标点符号和一些其他符号

字符	描述

[ABC]	匹配 [...] 中的所有字符，例如 [aeiou] 匹配字符串 "google runoob taobao" 中所有的 e o u a 字母。 
[^ABC]	匹配除了 [...] 中字符的所有字符，例如 [^aeiou] 匹配字符串 "google runoob taobao" 中除了 e o u a 字母的所有字母。
[A-Z]	[A-Z] 表示一个区间，匹配所有大写字母，[a-z] 表示所有小写字母。 
.	匹配除换行符（\n、\r）之外的任何单个字符，相等于 [^\n\r]。

[\s\S]	匹配所有。\s 是匹配所有空白符，包括换行，\S 非空白符，不包括换行。 
\w	匹配字母、数字、下划线。等价于 [A-Za-z0-9_]

2、非打印字符

非打印字符也可以是正则表达式的组成部分。下表列出了表示非打印字符的转义序列

字符	描述
\cx	匹配由x指明的控制字符。例如， \cM 匹配一个 Control-M 或回车符。x 的值必须为 A-Z 或 a-z 之一。否则，将 c 视为一个原义的 'c' 字符。
\f	匹配一个换页符。等价于 \x0c 和 \cL。
\n	匹配一个换行符。等价于 \x0a 和 \cJ。
\r	匹配一个回车符。等价于 \x0d 和 \cM。
\s	匹配任何空白字符，包括空格、制表符、换页符等等。等价于 [\f\n\r\t\v]。注意 Unicode 正则表达式会匹配全角空格符。
\S	匹配任何非空白字符。等价于 [^ \f\n\r\t\v]。
\t	匹配一个制表符。等价于 \x09 和 \cI。

\v	匹配一个垂直制表符。等价于 \x0b 和 \cK。

3、特殊字符

所谓特殊字符，就是一些有特殊含义的字符，如上面说的 `runoo*b` 中的 `*`，简单的说就是表示任何字符串的意思。如果要查找字符串中的 `*` 符号，则需要对 `*` 进行转义，即在其前加一个 `\`，`runo\*ob` 匹配字符串 runo*ob。

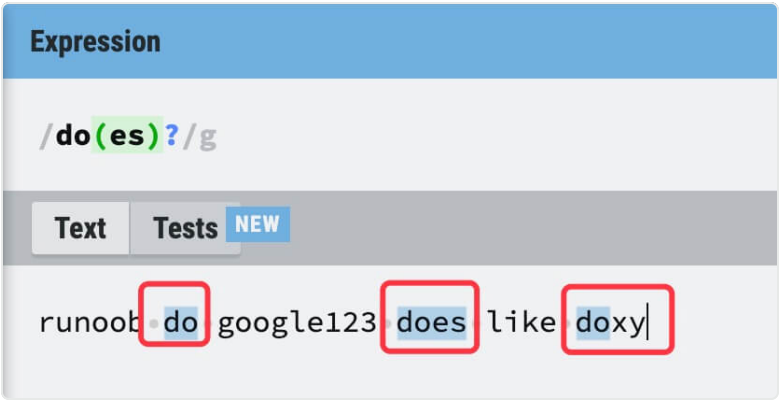
许多元字符要求在试图匹配它们时特别对待。若要匹配这些特殊字符，必须首先使字符"转义"，即，将反斜杠字符`\` 放在它们前面。下表列出了正则表达式中的特殊字符：

特别字符	描述
$	匹配输入字符串的结尾位置。如果设置了 RegExp 对象的 Multiline 属性，则 $ 也匹配 '\n' 或 '\r'。要匹配 $ 字符本身，请使用 \$。
( )	标记一个子表达式的开始和结束位置。子表达式可以获取供以后使用。要匹配这些字符，请使用 \(和 \)。
*	匹配前面的子表达式零次或多次。要匹配 * 字符，请使用 *。
+	匹配前面的子表达式一次或多次。要匹配 + 字符，请使用 \+。
.	匹配除换行符 \n 之外的任何单字符。要匹配 . ，请使用 \. 。
[	标记一个中括号表达式的开始。要匹配 [，请使用 \[。
?	匹配前面的子表达式零次或一次，或指明一个非贪婪限定符。要匹配 ? 字符，请使用 \?。
\	将下一个字符标记为或特殊字符、或原义字符、或向后引用、或八进制转义符。例如， 'n' 匹配字符 'n'。'\n' 匹配换行符。序列 '\\' 匹配 "\"，而 '\(' 则匹配 "("。
^	匹配输入字符串的开始位置，除非在方括号表达式中使用，当该符号在方括号表达式中使用时，表示不接受该方括号表达式中的字符集合。要匹配 ^ 字符本身，请使用 \^。
{	标记限定符表达式的开始。要匹配 {，请使用 \{。
|	指明两项之间的一个选择。要匹配 |，请使用 \\|。

4、限定符

限定符用来指定正则表达式的一个给定组件必须要出现多少次才能满足匹配。有 `*` 或 `+` 或 `?` 或 `{n}` 或 `{n,}` 或 `{n,m}` 共6种。

正则表达式的限定符有：

字符	描述
*	匹配前面的子表达式零次或多次。例如，`zo*` 能匹配 "z" 以及 "zoo"。`*` 等价于 `{0,}`。
+	匹配前面的子表达式一次或多次。例如，`zo+` 能匹配 "zo" 以及 "zoo"，但不能匹配 "z"。`+` 等价于 `{1,}`。
?	匹配前面的子表达式零次或一次。例如，`do(es)?` 可以匹配 "do" 、 "does"、 "doxy" 中的 "do" 。`?` 等价于 `{0,1}`。 
{n}	n 是一个非负整数。匹配确定的 n 次。例如，`o{2}` 不能匹配 "Bob" 中的 o，但是能匹配 "food" 中的两个 o。
{n,}	n 是一个非负整数。至少匹配n 次。例如，`o{2,}` 不能匹配 "Bob" 中的 o，但能匹配 "foooood" 中的所有 o。`o{1,}` 等价于 `o+`。`o{0,}` 则等价于 `o*`。
{n, m}	m 和 n 均为非负整数，其中 n <= m。最少匹配 n 次且最多匹配 m 次。例如，`o{1,3}` 将匹配 "fooooood" 中的前三个 o。`o{0,1}` 等价于 `o?`。请注意在逗号和两个数之间不能有空格。

5、定位符

定位符使您能够将正则表达式固定到行首或行尾。它们还使您能够创建这样的正则表达式，这些正则表达式出现在一个单词内、在一个单词的开头或者一个单词的结尾。

定位符用来描述字符串或单词的边界，`^` 和 `$` 分别指字符串的开始与结束，`\b` 描述单词的前或后边界，`\B` 表示非单词边界。

正则表达式的定位符有：

字符	描述
^	匹配输入字符串开始的位置。如果设置了 RegExp 对象的 Multiline 属性，^ 还会与 \n 或 \r 之后的位置匹配。
$	匹配输入字符串结尾的位置。如果设置了 RegExp 对象的 Multiline 属性，$ 还会与 \n 或 \r 之前的位置匹配。
\b	匹配一个单词边界，即字与空格间的位置。
\B	非单词边界匹配。

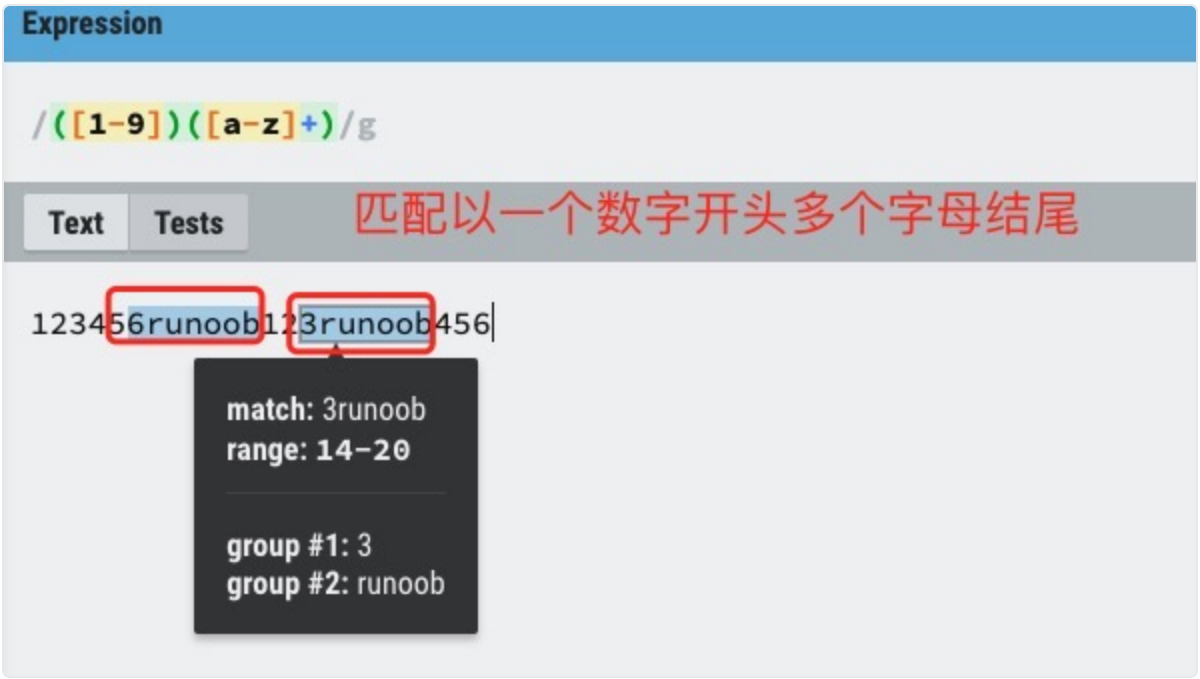
- **注意**：不能将限定符与定位符一起使用。由于在紧靠换行或者单词边界的前面或后面不能有一个以上位置，因此不允许诸如 `^*` 之类的表达式。
- 若要匹配一行文本开始处的文本，请在正则表达式的开始使用 `^` 字符。不要将 `^` 的这种用法与中括号表达式内的用法混淆。
- 若要匹配一行文本的结束处的文本，请在正则表达式的结束处使用 `$` 字符。

6、选择

①、括号 `()`

用圆括号 `()` 将所有选择项括起来，相邻的选择项之间用 `|` 分隔。

`()` 表示捕获分组，`()` 会把每个分组里的匹配的值保存起来， 多个匹配值可以通过数字 n 来查看(**n** 是一个数字，表示第 n 个捕获组的内容)



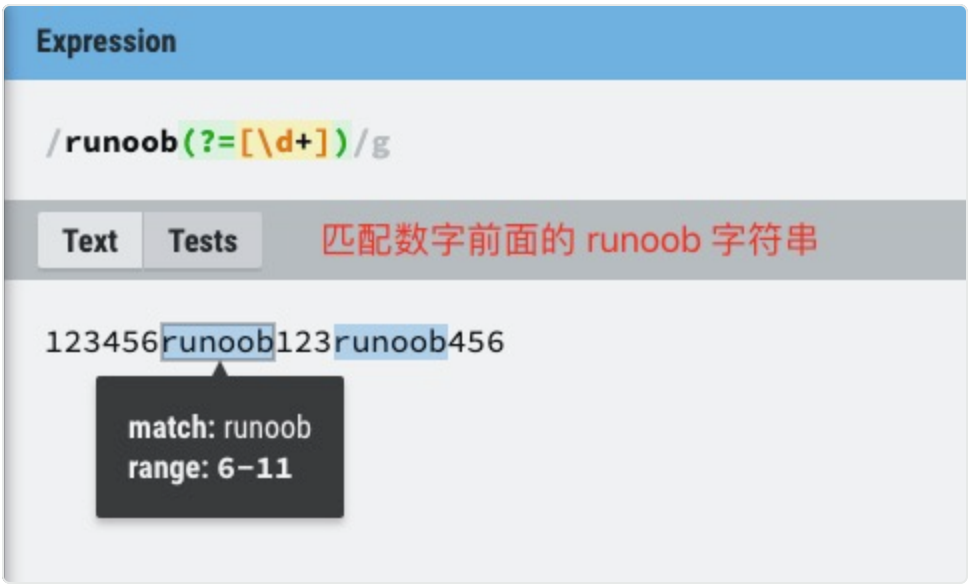
但用圆括号会有一个副作用，使相关的匹配会被缓存，此时可用 `?:` 放在第一个选项前来消除这种副作用。

其中 `?:` 是非捕获元之一，还有两个非捕获元是 `?=` 和 `?!`，这两个还有更多的含义，前者为正向预查，在任何开始匹配圆括号内的正则表达式模式的位置来匹配搜索字符串，后者为负向预查，在任何开始不匹配该正则表达式模式的位置来匹配搜索字符串。

---

②、 `?=` 、 `?<=` 、 `?!` 、 `?<!` 的使用区别

1. `exp1(?=exp2)` ： 查找 exp2 前面的 exp1。



2. `(?<=exp2)exp1` ： 查找 exp2 后面的 exp1。

3. `exp1(?!exp2)` ： 查找后面不是 exp2 的 exp1。

4. `(?<!exp2)exp1` ： 查找前面不是 exp2 的 exp1。

7、反向引用

对一个正则表达式模式或部分模式两边添加圆括号将导致相关匹配存储到一个临时缓冲区中，所捕获的每个子匹配都按照在正则表达式模式中从左到右出现的顺序存储。缓冲区编号从 1 开始，最多可存储 99 个捕获的子表达式。每个缓冲区都可以使用 `\n` 访问，其中 n 为一个标识特定缓冲区的一位或两位十进制数。

可以使用非捕获元字符 `?:`、`?=` 或 `?!` 来重写捕获，忽略对相关匹配的保存。

三、

四、

五、

六、

七、

八、

九、

十、

十一、

十二、

十三、

十四、

十五、

十六、

十七、

十八、

十九、

二十、

1、

2、

3、

4、

5、

6、

7、

8、

9、

①、

②、

③、

④、

⑤、

⑥、

⑦、

⑧、

⑨、

⑩、

Ⅰ、

Ⅱ、

Ⅲ、

Ⅳ、

Ⅴ、

Ⅵ、

Ⅶ、

Ⅷ、

Ⅸ、

Ⅹ、